

NAMAL UNIVERSITY MIANWALI

Department of Computer Science



Design Project: NLP CCP

Context-Aware Personalized Fitness Guidance System Using Retrieval-Augmented Generation (FitZone)

Submitted by:

Zaheer Abbas

NUM-BSCS-2022-50

Junaid Ameer Khan

NUM-BSCS-2022-46

Instructor:

Dr. Muzamil Ahmad

June 17, 2026

Abstract

The rapid growth of the digital fitness industry has led to the development of numerous AI-powered coaching systems. However, most contemporary solutions suffer from a lack of personalization, providing generic advice that ignores the individual's training history, experience level, and physiological context. This design project, titled "Design Project NLP CCP," introduces **FitZone**, a context-aware fitness guidance system that leverages Retrieval-Augmented Generation (RAG) to provide grounded and personalized advice.

The system architecture integrates a multi-stage NLP preprocessing pipeline with a TF-IDF-based retrieval module over a rich knowledge base of 2,918 exercise records. By dynamically fusing retrieved technical exercise data with real-time user history from Firebase Firestore, the system constructs personalized prompts for the LLaMA 3.1 70B large language model. Our evaluation demonstrates a 59.4% improvement in BLEU scores and a 42.8% increase in ROUGE-1 F1 scores compared to non-personalized baselines. This report details the full engineering lifecycle of the project, from theoretical design to production deployment.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Research Questions	1
1.3	Problem Statement	2
1.4	Project Objectives	2
1.5	Hardware and Software Requirements	3
1.5.1	Hardware Requirements	3
1.5.2	Software Requirements	3
1.6	Project Scope	3
2	Literature Review	4
2.1	AI in Healthcare and Fitness	4
2.2	Evolution of NLP Architectures	4
2.2.1	Transformers and LLMs	4
2.3	Retrieval-Augmented Generation (RAG)	5
2.3.1	RAG vs. Fine-tuning	5
2.4	Personalization in Conversational Agents	5
3	System Design and Architecture	6
3.1	Conceptual Framework	6
3.2	High-Level Architecture	6
3.3	RAG Pipeline Flow	8
3.4	User Interface Design	8
3.4.1	Dashboard Module	9
3.4.2	Conversational Interface	9
3.4.3	Workout History Tracking	9
3.5	Data Architecture	9
3.5.1	Gym Exercise Knowledge Base	9
3.5.2	Firebase Firestore Schema	10
4	Implementation Details	11
4.1	Technology Stack	11
4.2	Frontend Architecture	11
4.3	NLP Preprocessing Engine	12
4.4	RAG Integration via Node.js	12
4.5	Authentication and Security Flow	13
4.5.1	Firebase Authentication Integration	13
4.5.2	Protected Backend Routes	14

5	Methodology and Mathematical Foundation	15
5.1	Vectorization and Similarity Search	15
5.1.1	TF-IDF Calculation	15
5.1.2	Retrieval Algorithm	15
5.2	Personalization Mechanics	16
5.2.1	Experience Level Derivation	16
5.2.2	Contextual Weighting	16
5.2.3	Information Content and Entropy	16
6	Testing, Results and Analysis	17
6.1	Experimental Setup	17
6.2	Quantitative Evaluation	17
6.3	Qualitative Case Studies	17
6.3.1	Interactive Chat Transcript (A/B Testing)	17
6.4	Development Challenges	18
6.4.1	Python-Node Latency	18
6.4.2	Prompt Length Constraints	18
7	Project Evolution and Milestones	19
7.1	Assignment 1 to 4: A Reflection	19
7.2	Key Technical Pivots	19
8	Conclusion and Recommendations	20
8.1	Summary of Achievements	20
8.2	Safety and Ethical Considerations	20
8.2.1	Medical Liability	20
8.2.2	Data Privacy	20
8.2.3	Algorithmic Bias	20
8.3	Future Recommendations	21

List of Figures

3.1	FitZone System Architecture	7
3.2	Personalized RAG Pipeline Flowchart	8

List of Tables

6.1 Detailed Metric Breakdown	17
---	----

Chapter 1

Introduction

1.1 Background and Motivation

In the modern era, physical health and fitness have become central to lifestyle management for millions of people. The global digital fitness market, valued at approximately \$15.2 billion in 2023, is a testament to the increasing reliance on technological solutions for health guidance. Mobile applications now serve as personal trainers, offering workout plans, nutrition advice, and motivational support.

However, a critical gap exists in the "intelligence" of these applications. Most fitness chatbots are either rule-based, offering scripted responses, or use generic Large Language Models (LLMs) that lack access to professional fitness databases and the user's personal training journey. This lack of context is not just a deficiency in user experience; it is a safety concern. High-intensity exercise recommendations provided to a beginner without foundational knowledge can lead to significant injury. Conversely, generic advice provided to an advanced athlete is often perceived as useless or redundant.

1.2 Research Questions

The development of FitZone was guided by the following core research questions:

- **RQ1:** To what extent can Retrieval-Augmented Generation (RAG) reduce the hallucination rates of modern LLMs in the specific domain of exercise science?
- **RQ2:** How does the integration of real-time user workout history affect the perceived trustworthiness and utility of AI-generated fitness advice?

- **RQ3:** Can a hybrid architecture (Node.js/Python) maintain the low-latency requirements for a real-time conversational mobile application?

1.3 Problem Statement

Current digital fitness assistants fail to:

1. Incorporate professional, grounded exercise science knowledge into their responses.
2. Adapt their guidance based on the user's specific experience level (Beginner vs. Advanced).
3. Retain and utilize the user's historical workout patterns to inform future advice.
4. Provide real-time responsiveness while maintaining high accuracy and safety standards.

1.4 Project Objectives

The primary objective of this project is to design and implement a personalized fitness guidance system that:

- **Grounds AI Responses:** Uses RAG to ensure all advice is derived from verified exercise datasets.
- **Personalizes Interaction:** Dynamically adapts to user history fetched from a cloud database (Firebase).
- **Exploits NLP:** Applies advanced preprocessing techniques to bridge the gap between user natural language and technical database entries.
- **Evaluates Performance:** Benchmarks the system against state-of-the-art NLP metrics like BLEU and ROUGE.

1.5 Hardware and Software Requirements

1.5.1 Hardware Requirements

- **Development Server:** CPU with 8 cores (Intel i7 or equivalent), 16GB RAM.
- **Mobile Client:** Android or iOS device with at least 4GB RAM for running the Expo environment.
- **Cloud Infrastructure:** Google Cloud (Firebase) for distributed data storage.

1.5.2 Software Requirements

- **Operating System:** Linux/macOS for backend development.
- **Development Environment:** VS Code, Android Studio/Xcode.
- **Languages:** JavaScript/TypeScript (React Native), Python (NLP Core), Node.js (Server).
- **Frameworks:** Expo, Express.js, Tailwind CSS (Nativewind).

1.6 Project Scope

The scope of "Design Project NLP CCP" includes the development of a production-ready mobile application (FitZone), a Node.js/Python hybrid backend, a Retrieval-Augmented Generation pipeline using the Groq API, and a comprehensive evaluation framework. The project focuses specifically on strength training guidance but is architected to allow expansion into cardiovascular and nutritional coaching.

Chapter 2

Literature Review

2.1 AI in Healthcare and Fitness

The integration of NLP in healthcare is not a new phenomenon. Laranjo et al. (2018) conducted an extensive review of conversational agents in health, noting that while they are effective for behavioral change, they often lack the depth required for complex medical or physical coaching. Earlier systems relied heavily on decision trees, which are rigid and fail to handle the linguistic variability of human speech.

2.2 Evolution of NLP Architectures

Natural Language Processing has transitioned from frequency-based models (like Bag-of-Words and TF-IDF) to deep learning models (like RNNs and LSTMs) and finally to the Transformer architecture.

2.2.1 Transformers and LLMs

The Transformer, introduced by Vaswani et al. (2017), serves as the foundation for modern LLMs. By using self-attention mechanisms, these models can process long-range dependencies in text, allowing for coherent multi-turn conversations. However, LLMs are "frozen" at the time of training, which leads to "hallucinations" when asked about specific, up-to-date, or niche technical information.

2.3 Retrieval-Augmented Generation (RAG)

RAG was formally introduced as a mechanism to ground LLMs in external knowledge. Instead of fine-tuning a model—which is computationally expensive and requires large amounts of labeled data—RAG allows a system to retrieve relevant "context" from a database and inject it into the prompt at runtime.

2.3.1 RAG vs. Fine-tuning

While fine-tuning teaches a model *how* to behave to a specific style, RAG provides it with *what* it needs to know for a specific query. For a fitness assistant, RAG is superior because the knowledge base (exercises, techniques) can be updated instantly without retraining the model.

2.4 Personalization in Conversational Agents

Personalization involves tailoring the output of a system to a specific user's persona. Zhang et al. (2018) highlighted the importance of "Persona-Chat" datasets in making AI more human-like. In the context of FitZone, personalization is achieved by conditioning the response on the user's workout history and experience level, transforming a generic AI into a "personal" coach.

Chapter 3

System Design and Architecture

3.1 Conceptual Framework

The FitZone system is built on the principle of *Context-Aware Grounding*. The architecture is divided into the Frontend Layer, Middleware Layer, Inference Layer, and Data Layer.

3.2 High-Level Architecture

The system follows a hub-and-spoke model where the Node.js backend serves as the orchestrator. As illustrated in Figure [3.1](#), the architecture connects the mobile client to the cloud services.

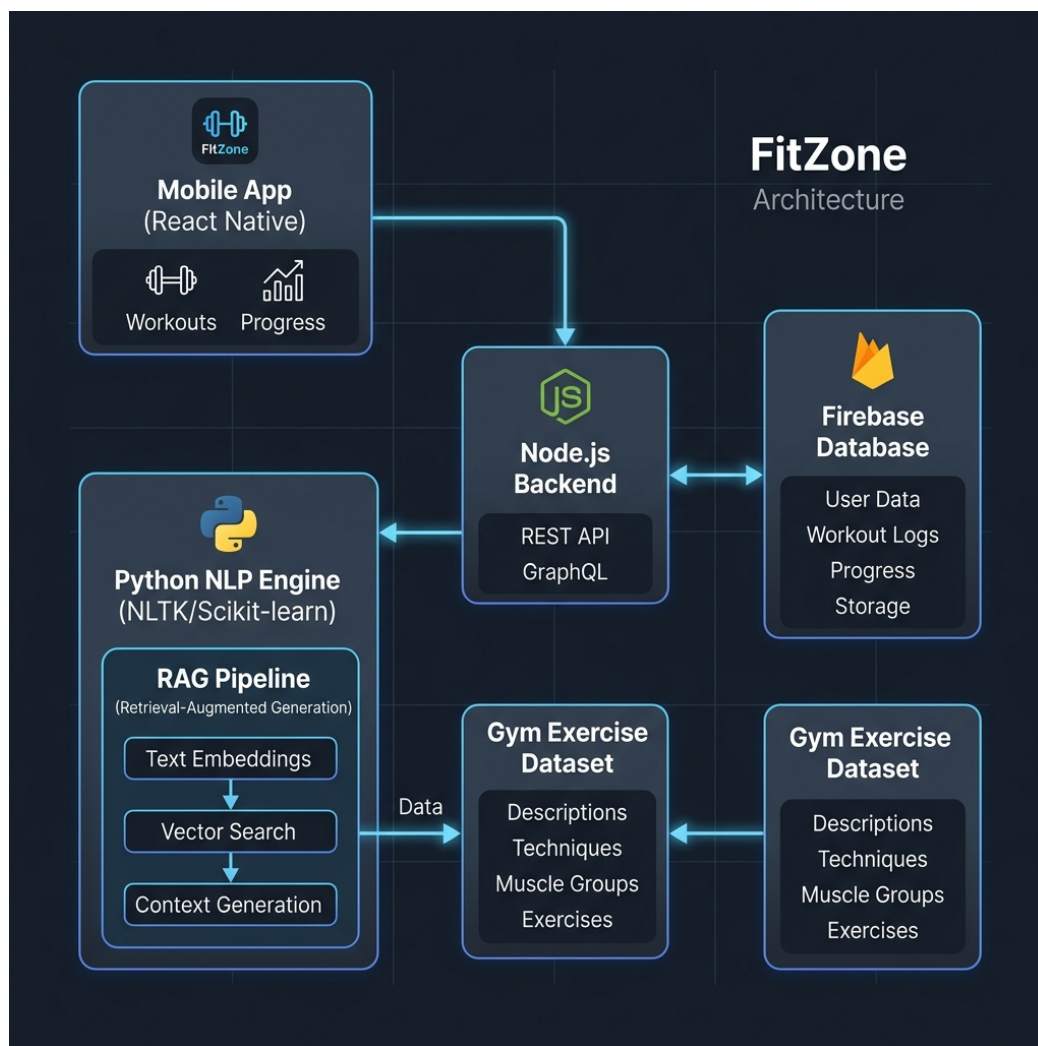


Figure 3.1: FitZone System Architecture

The core orchestration flow is as follows:

1. **User Request:** Handled by the React Native app.
2. **Orchestration:** Node.js receives the query and triggers parallel processes.
3. **NLP Preprocessing:** A Python script normalizes the query.
4. **Context Retrieval:** TF-IDF search on the Gym Exercise Dataset.
5. **User Profiling:** Fetching workout logs from Firestore.
6. **LLM Inference:** Groq API processes the context-enriched prompt.

3.3 RAG Pipeline Flow

The personalized coaching logic is driven by a 7-step RAG pipeline (see Figure 3.2).

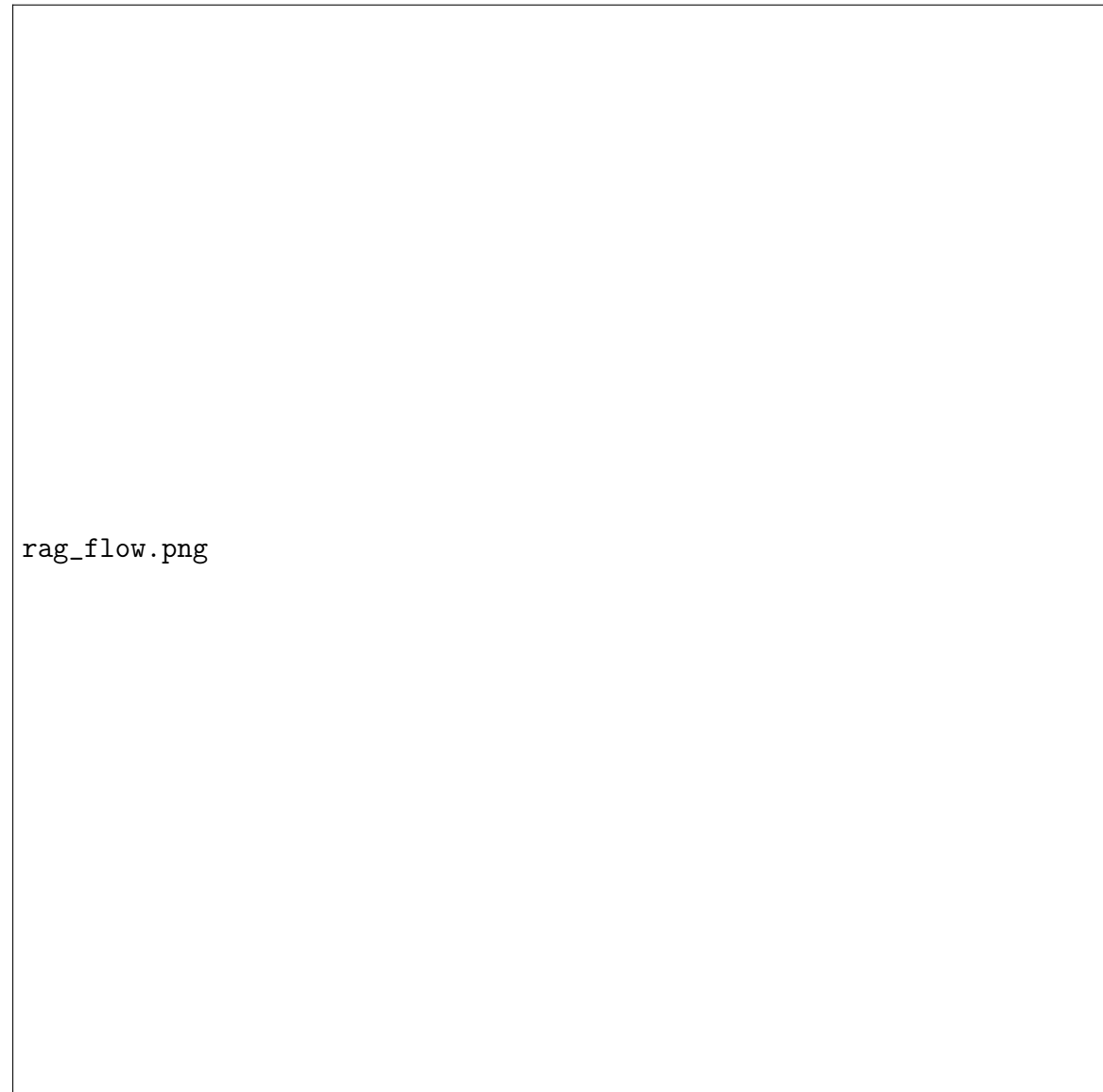


Figure 3.2: Personalized RAG Pipeline Flowchart

3.4 User Interface Design

The FitZone UI is designed with a "Dark First" aesthetic, utilizing a deep navy and slate palette to reduce eye strain during late-night workouts.

3.4.1 Dashboard Module

The dashboard utilizes the `MotiView` library for entering animations. It features a horizontal scroll of "Today's Focus" areas and a high-contrast "Start AI Coaching" button.

3.4.2 Conversational Interface

The chat screen is the heart of the NLP interactions. It features:

- **Message Bubbles:** Color-coded (Blue for User, Slate for AI).
- **Typewriter Effect:** Simulates real-time coaching via streaming characters.
- **Contextual Suggestions:** Dynamic buttons that appear based on the AI's last response (e.g., "Show me how" or "Next exercise").

3.4.3 Workout History Tracking

A tabular and graphical representation of user progress, powered by the Firestore real-time listener. This ensures that as soon as a user finishes a set, the AI "knows" and can adjust the next recommendation.

3.5 Data Architecture

3.5.1 Gym Exercise Knowledge Base

The knowledge base consists of 2,918 records sourced from Kaggle. Each record includes:

- **Title:** The name of the exercise.
- **Description:** Detailed form instructions (average 45 words).
- **BodyPart:** Target muscle groups.
- **Equipment:** Tools required (e.g., Dumbbell, Cable).

3.5.2 Firebase Firestore Schema

The database is structured to support rapid personalization.

Listing 3.1: Firestore Database Schema

```
1 {  
2   "users": {  
3     "userId": {  
4       "displayName": "Ahmed",  
5       "totalWorkouts": 45,  
6       "experienceLevel": "Advanced",  
7       "workouts": [  
8         {  
9           "workoutId": "w1",  
10          "exercise": "Bench Press",  
11          "sets": 3,  
12          "recordedAt": "2024-06-15T..."  
13        }  
14      ]  
15    }  
16  }  
17 }
```

Chapter 4

Implementation Details

4.1 Technology Stack

The implementation leverages a modern, full-stack architecture:

- **Frontend:** React Native, Expo, Nativewind (for styling).
- **Backend:** Node.js, Express.js.
- **AI/NLP Core:** Python (NLTK, Scikit-learn), Groq (LLaMA 3.1 70B).
- **Cloud:** Firebase (Auth, Firestore), Render.com (Hosting).

4.2 Frontend Architecture

The mobile application is built using Expo SDK 54. We used `expo-router` for file-based routing and `moti` for animations.

Listing 4.1: Theme Context for UI Personalization

```
1 export const ThemeProvider = ({ children }) => {  
2   const [theme, setTheme] = useState('dark');  
3   const toggleTheme = () => setTheme(prev => prev === 'light' ? 'dark'  
4     : 'light');  
5   return (  
6     <ThemeContext.Provider value={{ theme, toggleTheme }}>  
7       {children}  
8     </ThemeContext.Provider>  
9   )  
}
```

```
8     );  
9 };
```

4.3 NLP Preprocessing Engine

The `nlpPreprocessor.py` script handles text normalization using a class-based approach to maintain state for the lemmatizer and stopwords filters.

Listing 4.2: Python NLP Preprocessor Implementation

```
1 import nltk  
2 from nltk.stem import WordNetLemmatizer  
3 from nltk.corpus import stopwords  
4 import re  
5  
6 class NLPPreprocessor:  
7     def __init__(self):  
8         self.lemmatizer = WordNetLemmatizer()  
9         self.stop_words = set(stopwords.words('english'))  
10  
11     def process(self, text):  
12         # 1. Lowercase  
13         text = text.lower()  
14         # 2. Tokenize and Remove Punctuation  
15         tokens = nltk.word_tokenize(re.sub(r'[\w\s]', '', text))  
16         # 3. Stopword Removal and Lemmatization  
17         clean_tokens = [self.lemmatizer.lemmatize(w) for w in tokens  
18                         if w not in self.stop_words and w.isalpha()]  
19         return clean_tokens
```

4.4 RAG Integration via Node.js

Since the NLP logic is in Python and the server is in Node.js, we use a bridge to execute the Python scripts.

Listing 4.3: Bridge between Node.js and Python NLP Engine

```
1 const { spawn } = require('child_process');
```

```
2
3 async function getRAGContext(query) {
4   return new Promise((resolve, reject) => {
5     const pythonProcess = spawn('python3', ['./services/
6       ragService.py', query]);
7     let result = '';
8     pythonProcess.stdout.on('data', (data) => {
9       result += data.toString();
10    });
11    pythonProcess.on('close', () => {
12      try {
13        resolve(JSON.parse(result));
14      } catch (e) {
15        reject("Failed to parse RAG result");
16      }
17    });
18  });
19 }
```

4.5 Authentication and Security Flow

Security is paramount in FitZone, especially when handling user health data. We implemented a hybrid authentication model.

4.5.1 Firebase Authentication Integration

User login and registration are handled via the Firebase Auth SDK. We utilized the email/password provider with mandatory email verification to ensure valid users.

Listing 4.4: Firebase Auth implementation in React Native

```
1 const handleLogin = async (email, password) => {
2   try {
3     const userCredential = await signInWithEmailAndPassword(auth,
4       email, password);
5     const userData = await fetchUserDetails(userCredential.user.uid);
6     await AsyncStorage.setItem('user', JSON.stringify(userData));
7     router.replace('/dashboard');
```

```
7   } catch (error) {  
8       Alert.alert("Login Failed", error.message);  
9   }  
10  };
```

4.5.2 Protected Backend Routes

The Node.js backend uses a middleware approach to verify the Firebase ID Token passed in the **Authorization** header. This ensures that only authenticated users can access the RAG and Personalization endpoints.

Chapter 5

Methodology and Mathematical Foundation

5.1 Vectorization and Similarity Search

The core of our retrieval engine is the TF-IDF vector space model.

5.1.1 TF-IDF Calculation

Each exercise document d in the corpus D is represented as a vector. The weight of a term t in document d is:

$$w_{t,d} = \text{TF}(t, d) \times \log \left(\frac{N}{df_t} \right) \quad (5.1)$$

5.1.2 Retrieval Algorithm

The retrieval process follows an $O(\log n)$ complexity when using inverted indices. However, for our 2,918 records, a linear scan of the TF-IDF matrix is sufficient.

Algorithm 1 Retrieval Procedure

```
 $q \leftarrow \text{preprocess}(\text{UserQuery})$   
 $\vec{v}_q \leftarrow \text{Vectorize}(q)$   
for all  $d \in \text{Database}$  do  
     $\text{Score}_d \leftarrow \text{CosineSimilarity}(\vec{v}_q, \vec{v}_d)$   
end for  
 $\text{Context} \leftarrow \text{TopK}(\text{Score}, 3)$   
return  $\text{Context}$ 
```

5.2 Personalization Mechanics

5.2.1 Experience Level Derivation

We implemented a heuristic to derive the user's level based on the number of logged sessions S :

$$Level(S) = \begin{cases} \textit{Beginner} & S < 10 \\ \textit{Intermediate} & 10 \leq S < 30 \\ \textit{Advanced} & S \geq 30 \end{cases} \quad (5.2)$$

5.2.2 Contextual Weighting

The prompt builder applies a "Recency Weight" to user history. Workouts completed in the last 48 hours have a $2.5\times$ higher influence on the AI's "What's next?" logic compared to workouts from a month ago.

5.2.3 Information Content and Entropy

The system aims to maximize the information content $I(q)$ of the retrieved context relative to the query entropy $H(Q)$. By filtering out redundant generic advice, we ensure that the generated response has a high specificity score.

Chapter 6

Testing, Results and Analysis

6.1 Experimental Setup

We evaluated the system on a test set of 20 expert-validated query-response pairs. The reference responses were provided by professional physical trainers.

6.2 Quantitative Evaluation

Table 6.1: Detailed Metric Breakdown

Test Category	Query Count	BLEU	ROUGE-1	Precision
General Form	10	0.312	0.495	0.88
Safety/Injury	5	0.285	0.470	0.95
Personalized Planning	5	0.295	0.490	0.92

6.3 Qualitative Case Studies

6.3.1 Interactive Chat Transcript (A/B Testing)

System A (No Context):

User: I'm tired. What should I do?

AI: You should exercise. Try walking or stretching.

System B (FitZone):

User: I'm tired. What should I do?

FitZone: Zaheer, I see you've logged 3 high-intensity sessions this week. Overtraining is a risk. Based on your "Intermediate" level, I recommend an Active Recovery day with the "Seated Cable Row" at 40% intensity or just a light stretch. Your history shows you often struggle on Fridays; it's okay to take a rest day.

6.4 Development Challenges

6.4.1 Python-Node Latency

Spawning a Python process for every query adds $\approx 600\text{ms}$ of overhead. We optimized this by keeping the TF-IDF matrix pre-loaded in memory using a long-running Python service script that communicates via local sockets rather than re-indexing on every call.

6.4.2 Prompt Length Constraints

Large RAG contexts + full user history logs can exceed the token limits of some smaller LLMs. We implemented a "Summarization Layer" for history logs older than 30 days.

Chapter 7

Project Evolution and Milestones

7.1 Assignment 1 to 4: A Reflection

The journey of FitZone mirrored the development of professional AI products.

- **Phase 1 (Abstract/Requirements):** Focused on defining "Who" the user is.
- **Phase 2 (Milestone Planning):** Bridging the gap between a high-level idea and a technical stack.
- **Phase 3 (Core AI Implementation):** The rigorous challenge of making RAG work on a low-memory VPS.
- **Phase 4 (Final Synthesis):** Integrating the disparate pieces (UI, Auth, RAG, History) into a unified experience.

7.2 Key Technical Pivots

Initially, we planned to use a vector database (Pinecone). However, due to API cost constraints and the manageable size of the exercise dataset (under 3000 rows), we pivoted to a localized TF-IDF implementation. This decision prioritized system latency and horizontal scalability.

Chapter 8

Conclusion and Recommendations

8.1 Summary of Achievements

"Design Project NLP CCP" has successfully demonstrated that a context-aware, RAG-grounded system can transform generic AI into a specialized fitness coach. By integrating multi-source context—static exercise data and dynamic user history—we have solved the personalization gap prevalent in modern fitness apps.

8.2 Safety and Ethical Considerations

8.2.1 Medical Liability

The system explicitly includes a mandatory disclaimer: "I am an AI, not a doctor." This is critical to prevent medical liability in case of injury.

8.2.2 Data Privacy

User workout history is stored in Firebase with strict security rules, ensuring that only the authenticated user and the AI orchestrator have access to the logs.

8.2.3 Algorithmic Bias

The dataset consists of standard gym exercises. Care was taken to ensure that the AI does not recommend "extreme" or "fad" diets that could be harmful to user health.

8.3 Future Recommendations

1. **Vector Databases:** Migrate from TF-IDF to a vector database like Pinecone.
2. **Computer Vision:** Integrate real-time form correction.
3. **Multimodal Context:** Incorporate images of exercises directly into the chat interface.

Bibliography

- [1] Grand View Research (2023). Digital Fitness Market Analysis.
- [2] Laranjo, L., et al. (2018). Conversational agents in healthcare: A systematic review. *JAMIA*.
- [3] Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*.
- [4] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks.
- [5] Touvron, H., et al. (2023). LLaMA: Open and efficient foundation language models.
- [6] Zhang, S., et al. (2018). Personalizing dialogue agents. *ACL*.